CS7NS6 Distributed Systems

Exercise 2: Implementing a Globally-Accessible Distributed Service

Clearway: Distributed Vehicle Capacity System

Trinity College Dublin Academic Year 2025/2026

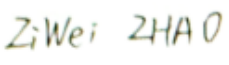
Signatures

Deepika Nag 

Ajinkya Taranekar 

Jai Nagle 

Xiaoxuan Duan 

Ziwei Zhao 

Contents

1. Requirements

1.1 Problem Statement

Road networks are capacity-constrained. Clearway treats road segment capacity as a *reservable resource*: drivers submit journey requests specifying origin, destination, departure time, and vehicle class; the system decomposes the route into ordered road segments, checks capacity atomically on each, and either approves or rejects the booking. The core technical challenge is a *concurrent reservation problem under distributed execution*: many drivers may simultaneously book overlapping routes, yet the sum of accepted reservations on any segment must never exceed physical capacity.

1.2 Functional Requirements

- **FR1 – Authentication.** Drivers and admins register and log in. Access tokens are verifiable locally by each service without a per-request call to the IAM service.
- **FR2 – Route Planning.** Given two road-network nodes, compute the shortest path as an ordered list of segments with per-segment traversal times.
- **FR3 – Atomic Capacity Reservation.** All-or-nothing reservation across every route segment. Minimum 60 minutes advance notice.
- **FR4 – Vehicle-Type Slot Weighting.** Fractional slot costs: car 1.0, motorcycle 0.5, van 1.5, HGV 3.0.
- **FR5 – Journey Lifecycle.** Seven-state machine: `PENDING` → `APPROVED`/`REJECTED` → `ACTIVE` → `COMPLETED`/`CANCELLED`/`EXPIRED`.
- **FR6 – Slot Release.** Release reserved slots asynchronously on terminal state.
- **FR7 – Push Notifications.** FCM push and in-app notifications for every lifecycle event.
- **FR8 – Segment Closures.** Enforcement officers and admins close/reopen road segments with time-bounded closure records (e.g. planned roadworks, incident management). Closures are checked atomically during reservation so no booking can be approved for a closed segment.
- **FR9 – Admin and Enforcement Traffic Map.** Live occupancy and closure overlay across the network. Enforcement officers use this view to identify overloaded corridors, verify which segments are currently closed, and cross-check active reservations against physical traffic. Admins additionally use it to force-cancel any journey and trigger emergency segment closures.
- **FR10 – Availability Query.** Query remaining capacity on any segment before committing to a booking.

1.3 Non-Functional Requirements

The most important non-functional property is *safety under concurrent write*: no combination of simultaneous booking requests may over-fill a segment. The second critical property is *event durability*: once a journey state change commits, downstream services must eventually process that event even if they are temporarily unavailable.

Performance targets are calibrated against the observed load profile: EU smoke tests showed a median booking latency of 1.1 s on a single-vCPU node; cross-region cells (US, APAC) add 80–120 ms per lock acquisition due to WAN leaseholder routing, compounding across multi-segment routes. At typical road-network peak (morning and evening rush hours) we modelled a concurrency spike of 20 simultaneous booking requests per cell - matching the K6 load scenario - as the dimensioning case. Segment capacity values (default 100 slots) are consistent with the Highway Capacity Manual benchmark of approximately 2,200 passenger-car-equivalents per lane per hour, scaled to the prototype's per-cell traffic model.

Category	Requirement
Consistency	`SERIALIZABLE` isolation + `SELECT FOR UPDATE`; no weaker level acceptable.
Event delivery	At-least-once via transactional outbox → Redis Streams → persistent consumer groups.
Idempotency	All mutating endpoints accept an `Idempotency-Key`; retries return cached responses.
Fault tolerance	No single service failure causes data loss or permanent inconsistency.
Scalability	Three independent regional cells (EU, US, APAC); no cross-region service calls.
Security	RS256 JWT; bcrypt cost-12; Nginx per-IP rate limiting.
Availability	Docker Swarm global mode; CockroachDB Raft tolerates one node failure per region; target uptime $\geq 99.5\%$ per cell.
Performance	EU p95 booking latency $< 5\,\text{s}$ at 20 concurrent VUs; cross-region p95 $< 12\,\text{s}$; Nginx p95 $< 100\,\text{ms}$ for read-only endpoints.
Business rules	60 min advance booking; 30 min cancel window; one active journey per driver; auto-expiry 30 min after departure.

Table 1: Non-functional requirements (quantitative targets derived from load-test baseline and Highway Capacity Manual load model)

2. Specification

2.1 Service Decomposition

The system comprises five independently-deployable microservices, each owning one CockroachDB schema. The decomposition follows ownership boundaries (auth, orchestration, capacity ledger, routing, notifications), so teams can evolve independently without schema collisions. The invariant "no segment is over-booked" is enforced entirely within the Capacity service, giving one authoritative place to audit reservation correctness.

#	Service	Port	Owner	Sole Responsibility
S1	IAM	8082	Deepika Nag	Credential issuance and JWKS publication
S2	Journey	8083	Ajinkya Taranekar	Booking orchestration, state machine, outbox relay
S3	Capacity	8081	Jai Nagle	Authoritative slot ledger; atomic reservation and release
S4	Map	8084	Xiaoxuan Duan	In-memory route graph; Dijkstra path computation
S5	Notification	8085	Ziwei Zhao	FCM push dispatch; in-app notification history

Table 2: Service registry - each service owns one CockroachDB schema

Communication mode is selected by timing semantics, not by framework preference. **Synchronous HTTP** is used when the caller must block for a result (Journey calls Map for a route, then Capacity for reservation approval). **Asynchronous Redis Streams** handle post-commit fan-out: notifications, slot release, and audit propagation. Redis was preferred over Kafka because it already existed in the stack and still provided the required ordered, consumer-group-aware recovery model via `XAUTOCLAIM` at prototype scale.

Service API Summary

All endpoints are prefixed `/api/v1` and require a `Authorization: Bearer <jwt>` header unless marked *public*. Every mutating endpoint requires an `Idempotency-Key` header.

Svc	Method	Path	Description
IAM Service (S1:8082)			
S1	POST	`/auth/register`	Register driver or admin; returns JWT + refresh token. *Public.*
S1	POST	`/auth/login`	Authenticate; returns JWT + refresh token. *Public.*
S1	POST	`/auth/refresh`	Exchange refresh token for new JWT. *Public.*
S1	POST	`/auth/logout`	Revoke all sessions for caller.
S1	GET	`/.well-known/jwks.json`	Public RSA key set for local JWT verification. *Public.*
S1	GET	`/auth/profile`	Return caller profile.
S1	POST	`/auth/promote`	Promote driver to admin (admin only).
Journey Service (S2:8083)			
S2	POST	`/journeys`	Create booking; triggers Route → Reserve → Persist flow. Returns `APPROVED` or `REJECTED`.
S2	GET	`/journeys`	List caller's journeys (driver) or all journeys (admin).
S2	GET	`/journeys/{id}`	Get single journey with segment breakdown.
S2	POST	`/journeys/{id}/activate`	Transition `APPROVED` → `ACTIVE`.
S2	POST	`/journeys/{id}/complete`	Transition `ACTIVE` → `COMPLETED`.
S2	POST	`/journeys/{id}/cancel`	Transition to `CANCELLED`; releases slots.
Capacity Service (S3:8081)			
S3	POST	`/capacity/reserve`	Reserve slots: single-region uses SERIALIZABLE transaction; multi-region routes through Saga coordinator with per-region sub-transactions and compensation on failure.
S3	POST	`/capacity/release`	Release reservation by ID (triggered via Redis event).
S3	GET	`/capacity/segments`	List all segments with `max_capacity`.
S3	GET	`/capacity/segments/occupancy`	Live occupancy per segment at given timestamp.
S3	POST	`/capacity/segments/{id}/close`	Create time-bounded closure (admin/enforcement).
S3	DELETE	`/capacity/segments/{id}/close/{cid}`	Remove closure record.
Map Service (S4:8084)			
S4	POST	`/routes/compute`	Dynamic routing: accepts {lat,lng} pair, calls OSRM (OpenStreetMap), caches route in DB, registers discovered segments in Capacity. Returns polyline + ordered segment list.
S4	GET	`/map/route`	Static Dijkstra on in-memory graph; requires named node IDs. Returns ordered segment list + traversal times.
S4	GET	`/map/nodes`	List all named road nodes loaded from `map.nodes` (used by origin/destination picker).
S4	GET	`/map/segments`	List all static segment topology metadata from `map.segments`.
S4	GET	`/map/search`	Geocode search via Nominatim (OpenStreetMap); returns matching places with coordinates.
S4	GET	`/map/traffic`	Live occupancy overlay: joins graph segments with real-time capacity data (admin/enforcement).
Notification Service (S5:8085)			
S5	GET	`/notifications`	List in-app notifications for caller.
S5	PUT	`/notifications/read-all`	Mark all notifications read.
S5	POST	`/notifications/device-token`	Register/update FCM device token.

Table 3: Service API summary - all endpoints are REST/JSON over HTTPS

2.2 Fault Behaviour

Failure handling was designed to preserve correctness first, then recover availability quickly. In practice this means "fail fast on dependencies, never commit partial booking state, replay from durable logs on restart."

Failed Component	Immediate Effect	Recovery Mechanism
IAM Service	New logins fail; no new tokens	All services validate JWTs from hourly-cached JWKS; sessions valid up to 1 h
Journey Service	New bookings fail; transactions roll back	Outbox relay re-reads `published=false` rows on restart
Capacity Service	Booking fails; no partial state	Client retries with same idempotency key
Map Service	Booking fails at route step	No state mutated; client retries
Notification	Events accumulate in Redis	`XAUTOCLAIM` reclaims idle entries after 60 s
Redis	Caching and Stream halted	Relay flushes buffered outbox rows on recovery
CockroachDB node	Cluster continues on 2-of-3 quorum	Raft elects new leader within seconds

Table 4: Fault behaviour and recovery per component

2.3 Limitations

Region granularity is coarse (`eu/us/apac`); the seeded road graph is Ireland-centric so almost all reservations fall in `eu`. Cross-region reservations are handled by the Saga coordinator (Section 3.6.3), which executes one serializable transaction per region and compensates already-reserved regions on partial failure; the cost is higher implementation complexity versus a single distributed transaction, but it avoids WAN-round-trip lock escalation across leaseholders. GPS tracking and payment are descoped. These choices were deliberate scope cuts: we prioritised reservation correctness, failure recovery, and end-to-end operability over feature breadth.

2.4 Network Partition and Split-Brain Behaviour

Under the CAP theorem this system explicitly chooses **consistency over availability** when a network partition occurs. The concrete behaviour is as follows.

Majority partition (2-of-3 nodes reachable). CockroachDB requires a Raft quorum of two replicas to accept any write. If one CockroachDB node loses network connectivity, the remaining two nodes still form a quorum and continue serving reads and writes without interruption. The isolated node enters a follower state; once the partition heals, it replays the Raft log to catch up and rejoins the cluster automatically. Drivers using the two healthy cells experience no disruption. Drivers whose HTTP request happens to hit the isolated node receive a `503 Service Unavailable`; they may retry with the same idempotency key and will succeed once the partition heals or once Nginx routes their request to a healthy cell.

Total partition (no majority - 1-1-1 split across EU, US, APAC). If all three cells lose inter-cell connectivity simultaneously, no cell can achieve a write quorum. All booking mutations block until the partition heals. Read-only endpoints (map search, occupancy query, notification list) continue to serve from local replicas using `AS OF SYSTEM TIME` follower reads, providing degraded but non-zero service. No data loss or split-brain write divergence occurs because CockroachDB never allows a write without quorum. This is the expected cost of strong global consistency: the system sacrifices write availability rather than risk two cells independently approving bookings that together violate segment capacity.

Partition healing. When network connectivity is restored, Raft leader election completes within seconds via heartbeat timeout detection (default 3 s). The previously-isolated node receives a snapshot or log replay from the current leader, depending on how far behind it has fallen, and becomes a voting replica again. Any transactions that were pending during the partition are either committed (if they reached quorum before isolation) or rolled back (if they did not). The idempotency layer ensures clients can safely retry rolled-back requests without risk of duplication.

Consistency invariant. At no point during any partition scenario can two separate cells simultaneously commit conflicting capacity reservations for the same segment. The `capacity.reservations` table's leaseholder is the single coordination point for all writes to a given partition; without quorum that leaseholder cannot commit. This is the fundamental guarantee the design relies on and which the chaos testing (Section 5.4) verified by isolating a CockroachDB node while simultaneous bookings continued.

3. Architecture and Design

3.1 System Architecture

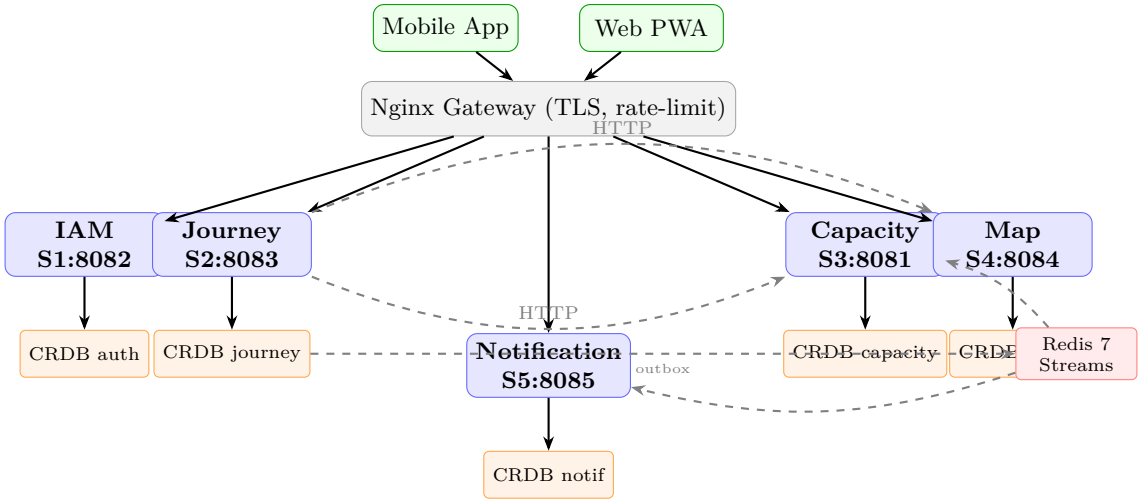


Figure 1: Per-cell architecture: Nginx gateway; five microservices communicating via HTTP (sync) and Redis Streams (async); each service owns a single CockroachDB schema

3.2 Cell-Based Deployment Model

Three independent regional cells exist: `europe-west1` (35.244.162.92), `us-east1` (35.227.198.68), and `asia-east1` (34.8.134.246), each on a GCP e2-medium VM (2 vCPU, 4 GB RAM, ~$25/month). A cell is a completely self-contained instance of all five services, one CockroachDB node, one Redis, and Nginx. The only inter-cell traffic is CockroachDB Raft replication on port 26257. This eliminates cross-region application latency and removes the single point of failure of a centralised deployment. The core rationale was first-principles latency and blast-radius control: a single global application deployment would force repeated WAN round-trips across service calls and make regional outages globally visible. The trade-off - no global capacity view - is acceptable because road segments are inherently regional.

3.3 Why CockroachDB

CockroachDB provides Raft consensus replication across three nodes; any two form a quorum; single-node failure is transparent within seconds. Its SERIALIZABLE isolation is equivalent in correctness to PostgreSQL's, maintained across a cluster. The reservation algorithm in Section 4 relies on this: two concurrent transactions attempting to reserve the last slot will not both commit - exactly one succeeds, the other receives error `40001` and retries. PostgreSQL wire-protocol compatibility meant no database-specific code changes were required. PostgreSQL was not rejected for correctness; it was rejected for operational fit in this scope: single-node HA would require extra orchestration, while CockroachDB gives distributed HA and serializable semantics in one engine.

Member failure detection and consensus. CockroachDB uses the Raft distributed consensus protocol for both replication and leader election. Each CockroachDB node sends periodic heartbeats to its peers every 200 ms. If a follower does not hear from the leader within the election timeout (approximately 3 s), it increments its *term counter*, transitions to *candidate* state, and broadcasts a `RequestVote` RPC to all other nodes. The first candidate to collect votes from a majority (two of three nodes) becomes the new leader for that term and immediately begins appending entries to the log. The previous leader, if it recovers, discovers a higher term on any RPC response and demotes itself to follower.

Writes require a quorum `AppendEntries` acknowledgement before the transaction coordinator receives a commit confirmation. This means a write that appears committed to the application is guaranteed to survive the loss of any single node: the log entry exists on at least two replicas. Range replicas rebalance automatically in the background after a node failure, restoring full three-way replication once the node recovers or is replaced. No manual operator intervention is required for single-node failures, which the chaos testing in Section 5.4 verified live against the production GCP cluster.

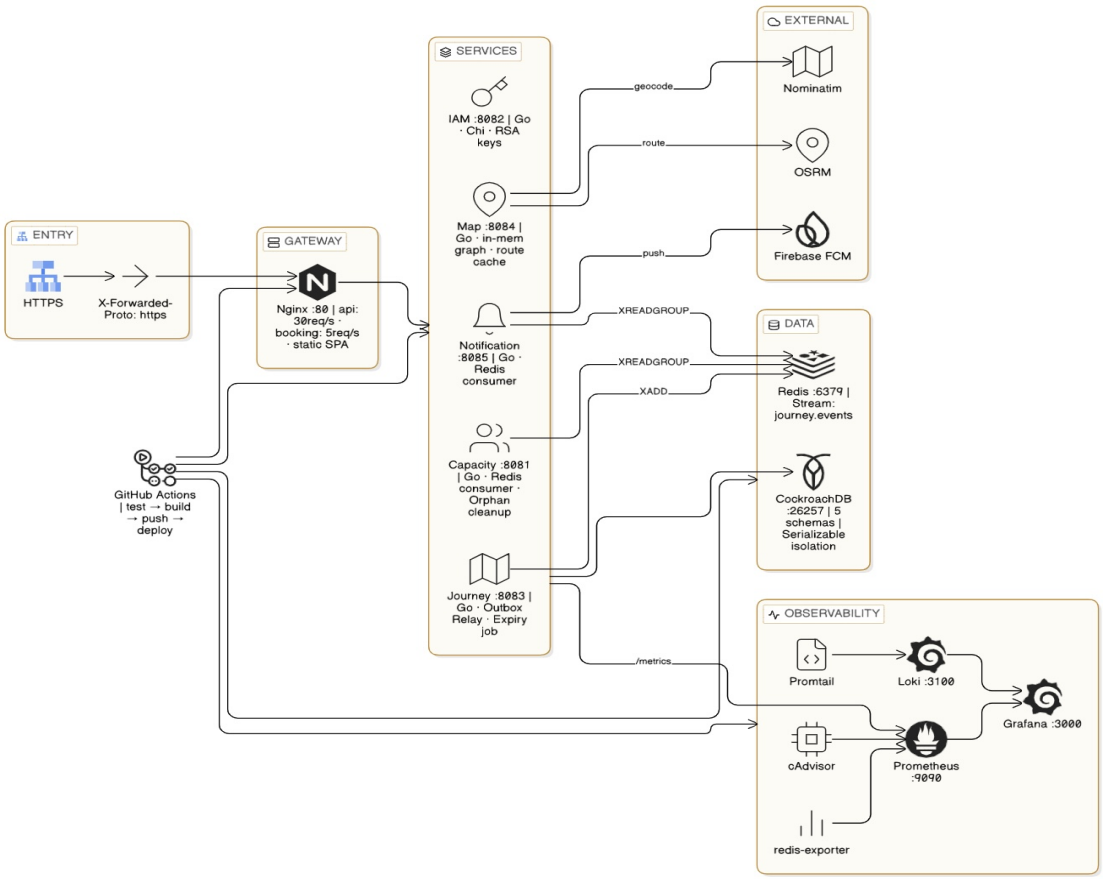


Figure 2: Infrastructure stack

3.4 Data Partitioning and Locality

Read-mostly reference tables (`capacity.segments`, `map.segments`, `map.nodes`) carry `LOCALITY GLOBAL` so every region keeps a local replica, avoiding WAN round-trips for data accessed on every booking but changed infrequently. Journey and reservation records are partitioned by `crdb_region` with leaseholders co-located with the origin gateway.

Cross-region locking carries a latency cost: every `SELECT FOR UPDATE` routes through the leaseholder. If a route spans multiple partitioned regions, the coordinator makes WAN round-trips to each leaseholder before acquiring all locks. This is the cost of strong atomicity across geography. Under CAP, this design prioritises consistency over availability under partition.

The Capacity service implements a full Saga coordinator for cross-region journeys. When `Reserve()` is called, it groups segments by `crdb_region` using a prefix convention (`US-*`→`us`, `JP-/AP-/SG-/CN-*`→`apac`, everything else →`eu`). If all segments fall within one region, the fast path (`doReserveTx`) executes a single SERIALIZABLE transaction as described in Section 3.6.3. If segments span multiple regions, `executeSaga()` is called: it creates a `PENDING` saga record, then executes one `doReserveSegmentsTx` per region in sorted order (`apac < eu < us`) to maintain deterministic lock ordering. On capacity failure in any region, all previously-committed regional transactions are compensated (reservations released). On crash recovery, a `PENDING` saga found on restart triggers compensation of any `RESERVED` steps before a fresh retry. The saga lifecycle uses four states: `PENDING`, `COMMITTED`, `COMPENSATED`, and `FAILED`, persisted in `capacity.reservation_sagas`.

3.5 Service Architecture

3.5.1 IAM Service

The IAM service holds an RSA-2048 private key and signs JWTs using RS256. The public key is served at `/.well-known/jwks.json`; all other services fetch and cache it hourly and verify tokens locally - no network call after the initial key fetch. A one-hour IAM outage is transparent to drivers with existing sessions. Refresh tokens are opaque 256-bit strings stored hashed; each use rotates the token. Passwords use bcrypt at cost factor 12.

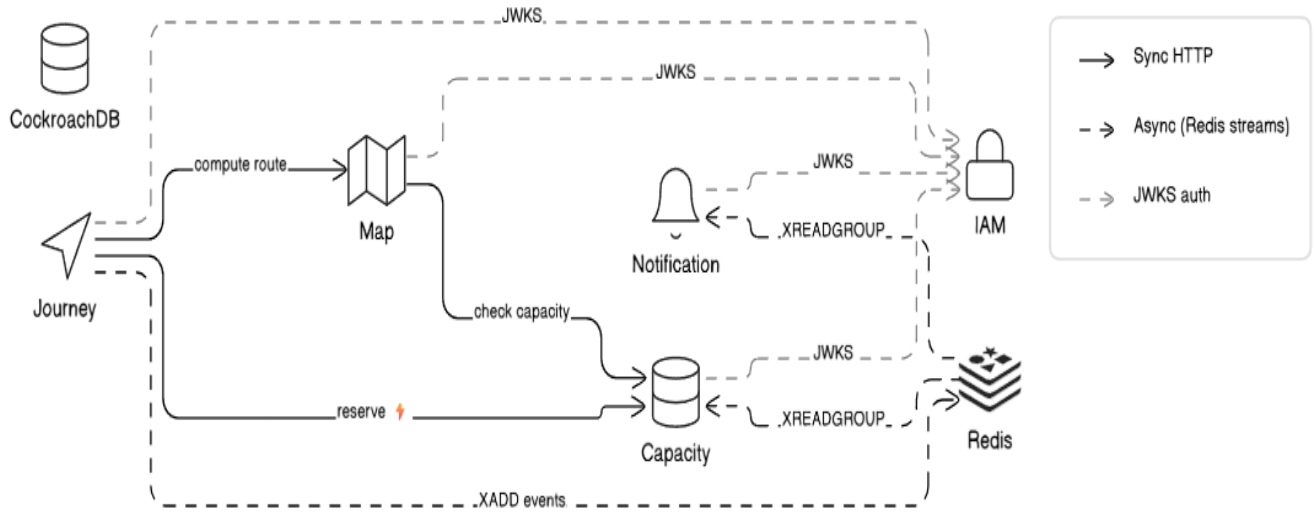


Figure 3: Service communication map: pentagon layout, sync and async links

This design intentionally avoids per-request IAM introspection. A synchronous auth call on every API request would create availability coupling and extra latency across all services; cached JWKS verification removes that dependency from the hot path.

3.5.2 Journey Service

The booking flow is ordered to minimise blast radius: (1) **Route** - call Map (read-only; failure leaves no state); (2) **Reserve** - call Capacity atomically (failure leaves no partial state); (3) **Persist** - write journey record, segment rows, and outbox entry in one transaction (the commit point; the outbox entry ensures the event reaches Redis regardless of subsequent crashes). Two background goroutines: **expiry job** (transitions stale journeys every 60 s) and **outbox relay** (polls every 1 s for `published=false` rows).

The ordering is deliberate: Journey does not persist booking state before capacity success, so no "journey exists but capacity never reserved" state can be committed. Pre-checks (single active journey per driver, 60-minute minimum lead time, idempotency key presence) run before downstream calls to fail early and reduce dependency load.

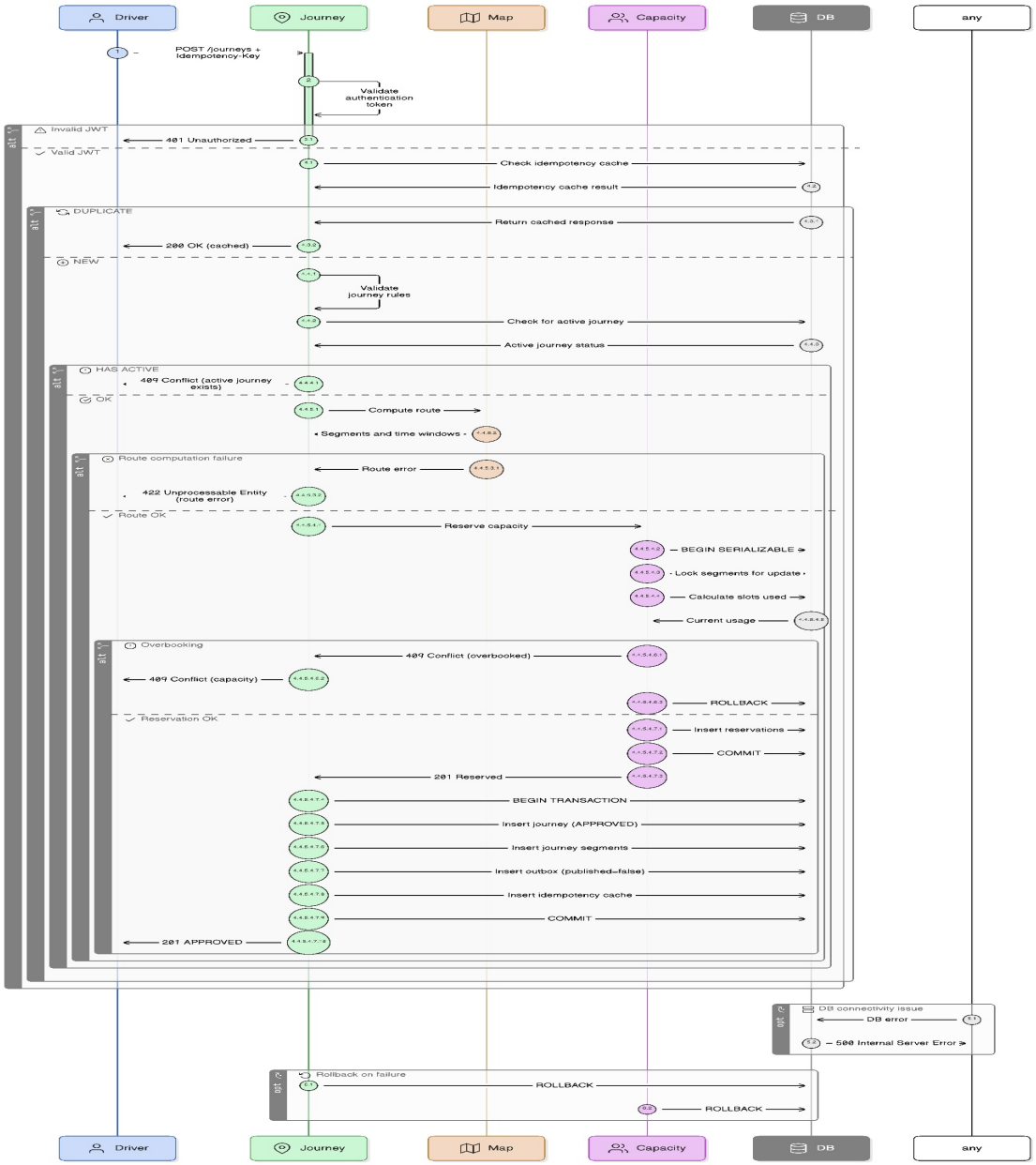


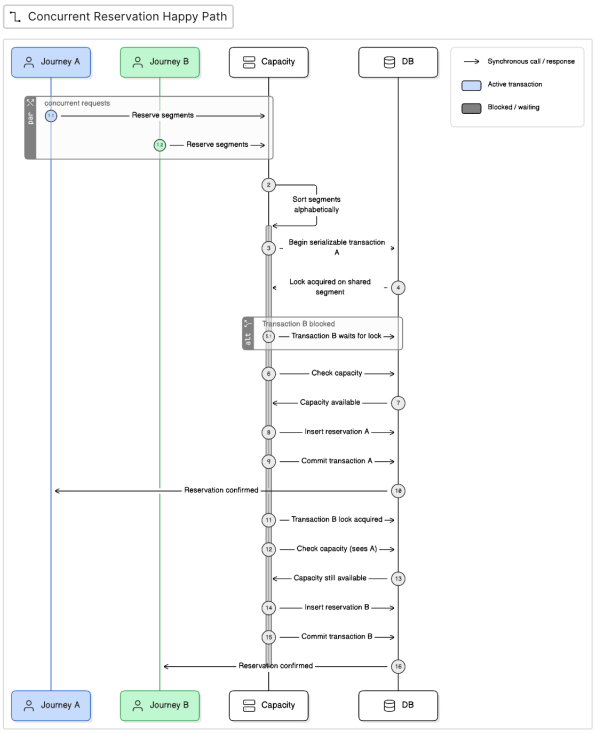
Figure 4: Core reservation steps

The Journey service wraps three downstream call sites with `gobreaker`: after 5 consecutive failures the breaker opens (fast-fail 502), preventing goroutine exhaustion from a slow downstream dependency. This converts slow cascading failure into explicit, quick dependency failure. Clients can retry safely with the same idempotency key.

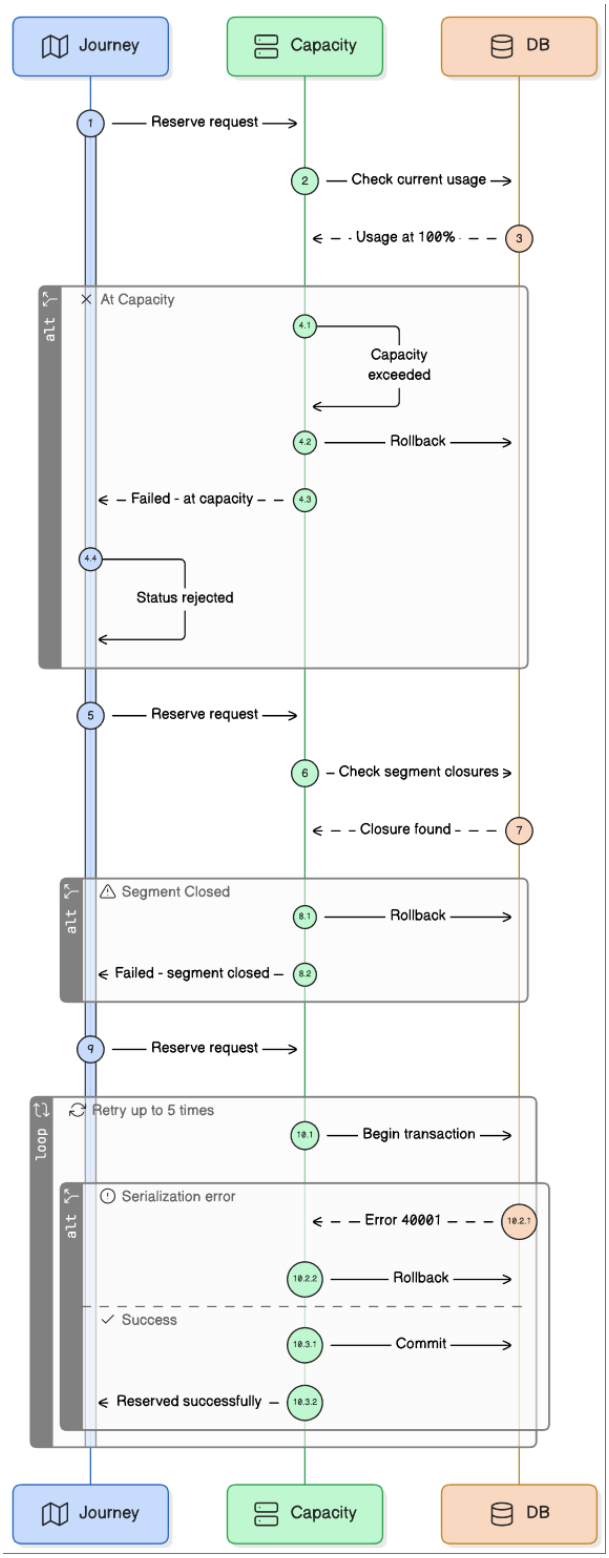
3.5.3 Capacity Service

The reservation algorithm executes in `BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE`. Segments are processed in *alphabetically-sorted order* to prevent deadlocks. Per-segment steps: `SELECT FOR UPDATE`; check active closures; sum occupancy; compare against `max_capacity`; proceed or roll back. Only if all segments pass are reservation rows inserted. Error `40001` triggers up to 5 retries; after 5 failures the service returns 503. Capacity uses **pessimistic locking**; journey state transitions use **optimistic locking** via a `version` integer.

The alphabetical lock order is the deadlock-avoidance rule: requests touching the same segment set acquire locks in a deterministic sequence, so one waits and re-checks after commit instead of both waiting on each other cyclically. The pessimistic versus optimistic split is also intentional: capacity conflicts are expected under load, while concurrent updates to one driver's single journey are comparatively rare.



(a) Concurrent happy path



(b) Failure scenarios

Figure 5: Capacity service: concurrent reservation and failure handling

3.5.4 Map and Notification Services

The **Map service** operates in two distinct routing modes selected at the API level.

Static node-based routing (`GET /api/v1/map/route`, `GET /api/v1/map/nodes`) uses a `GraphStore` that loads the road topology from `map.nodes` and `map.segments` in CockroachDB at startup into a concurrent in-memory snapshot (protected by a `sync.RWMutex`). All Dijkstra path computation for named nodes is answered from this in-memory structure in sub-millisecond time. If the database is unavailable at startup, `GraphStore` falls

back to a hardcoded 10-node Dublin network so the service remains operational.

Dynamic coordinate-based routing (`POST /api/v1/routes/compute`) accepts arbitrary GPS coordinates and resolves them against two external OpenStreetMap services. A **Nominatim** client geocodes place-name search queries (`nominatim.openstreetmap.org`). An **OSRM** client (`router.project-osrm.org`) computes the real driving route via the OSRM HTTP API (`/route/v1/driving/{coords}?overview=full&geometries=geojson&steps=true`). The response includes distance, duration, step-by-step road names and references, and a full GeoJSON polyline. OSRM steps are collapsed by road reference into logical segments (`osrm_{normalised_road_ref}`). The full route is persisted in CockroachDB (`map.routes`, `map.route_segments`, `map.intercity_segments`, `map.places`) so subsequent requests for the same coordinate pair are served from cache without an OSRM round-trip. After every successful route, newly-discovered segment IDs are automatically registered in the Capacity service via `ensureCapacitySegments()` so capacity tracking is always consistent with the routes actually returned.

Segment IDs are authoritative in Map; the Capacity service's table is kept consistent via this registration mechanism. Journey computes per-segment cascading time windows from the route and sends those to Capacity, so admission control is evaluated on when the driver will actually occupy each segment, not on route-level averages.

The **Notification service** consumes the `journey.events` Redis Stream; per event it inserts a `PENDING` notification, fetches the FCM token, calls the Firebase Admin SDK, and acknowledges with `XACK`. A `UNIQUE(event_id)` constraint makes redelivered events idempotent. `XAUTOCLAIM` reclaims idle entries after 60 s.

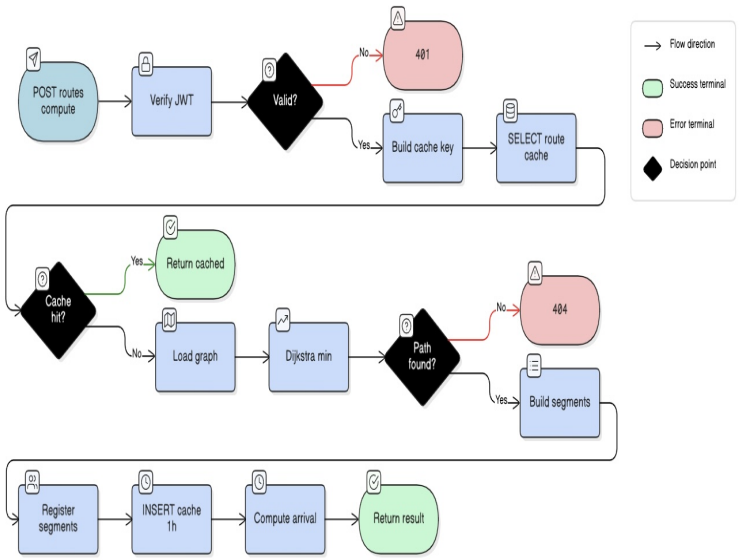


Figure 6: Map route computation

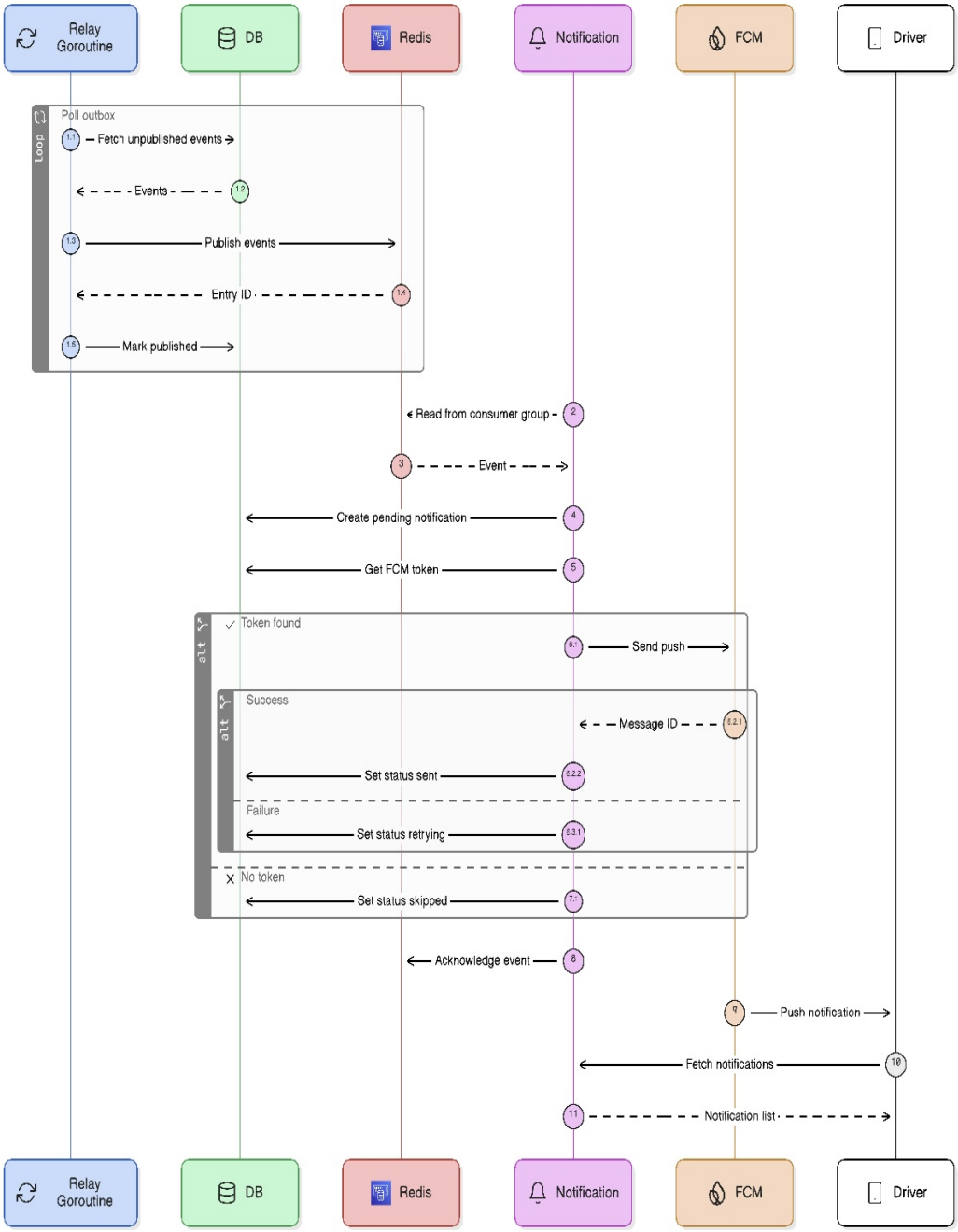


Figure 7: Outbox relay and notification delivery

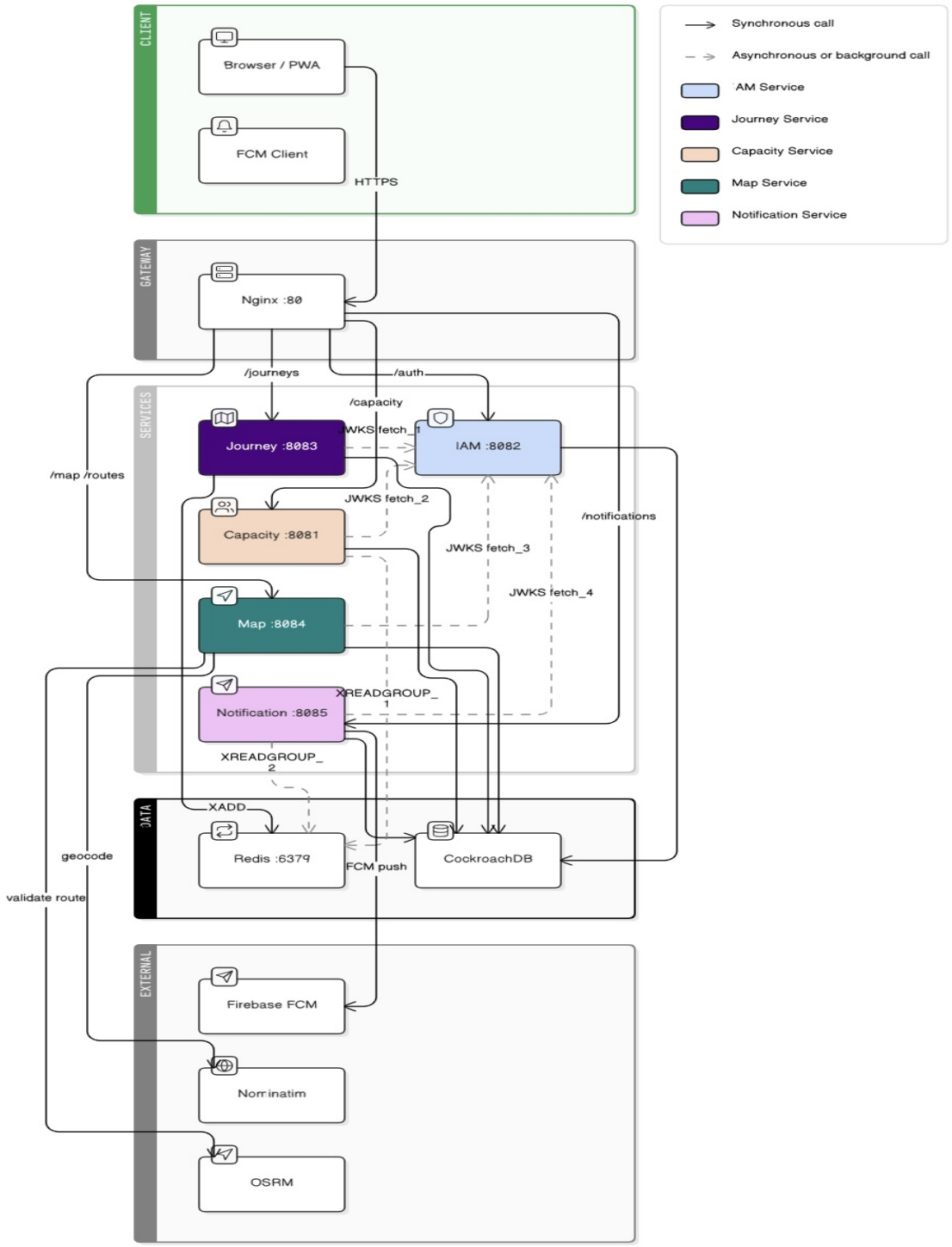
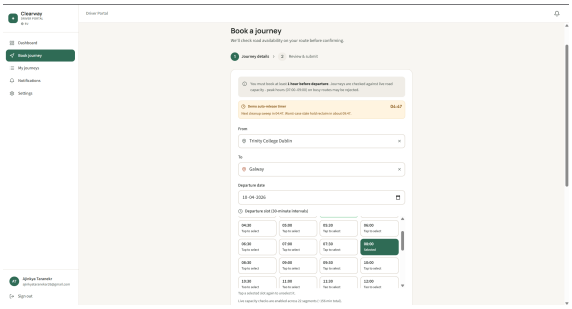


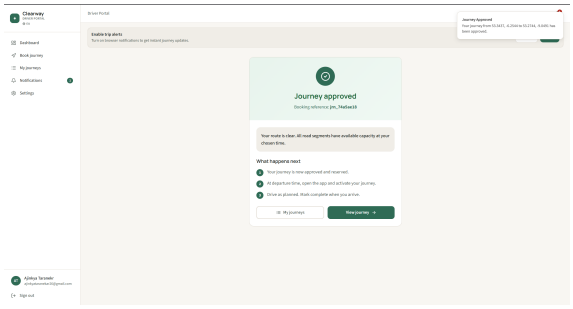
Figure 8: Redis event pipeline

3.6 Frontend

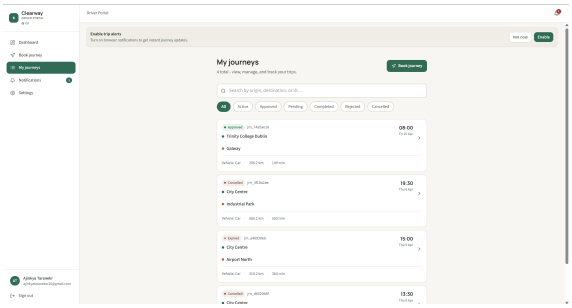
The React 18 PWA is served as a static build embedded inside Nginx. Application state lives in a central `AppContext` (user, tokens, journey list, notification badge). Below are the key UI screenshots from the `screenshots/` folder (real project output).



(a) Book journey

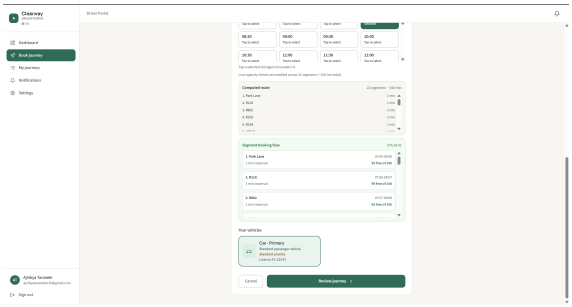


(b) Approved

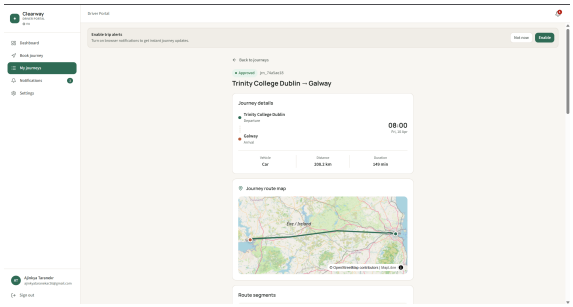


(c) Journey list

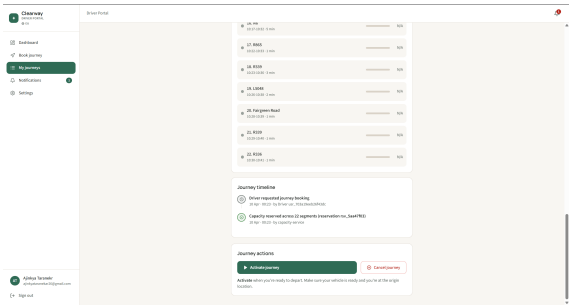
Figure 9: Driver flow: booking form, approval, and journey list



(a) Route preview

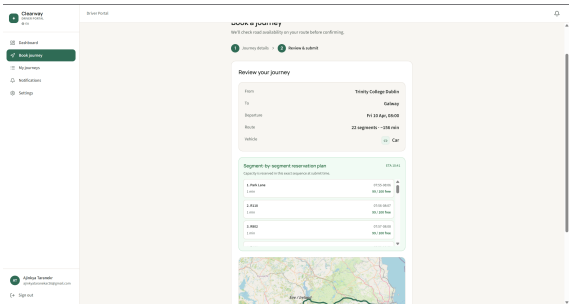


(b) Journey detail

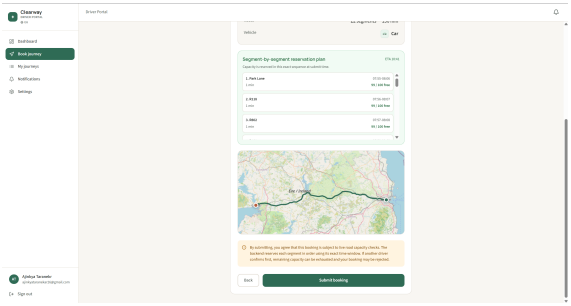


(c) Segment breakdown

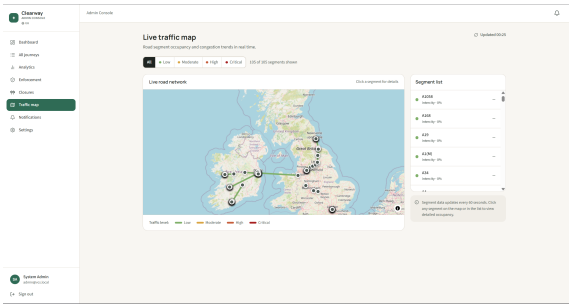
Figure 10: Driver flow continuation: preview and detailed journey views



(a) Admin journeys



(b) Admin review



(c) Live traffic

Figure 11: Admin tools: journey list, review panel, and live traffic map

4. Implementation

4.1 Transactional Outbox Pattern

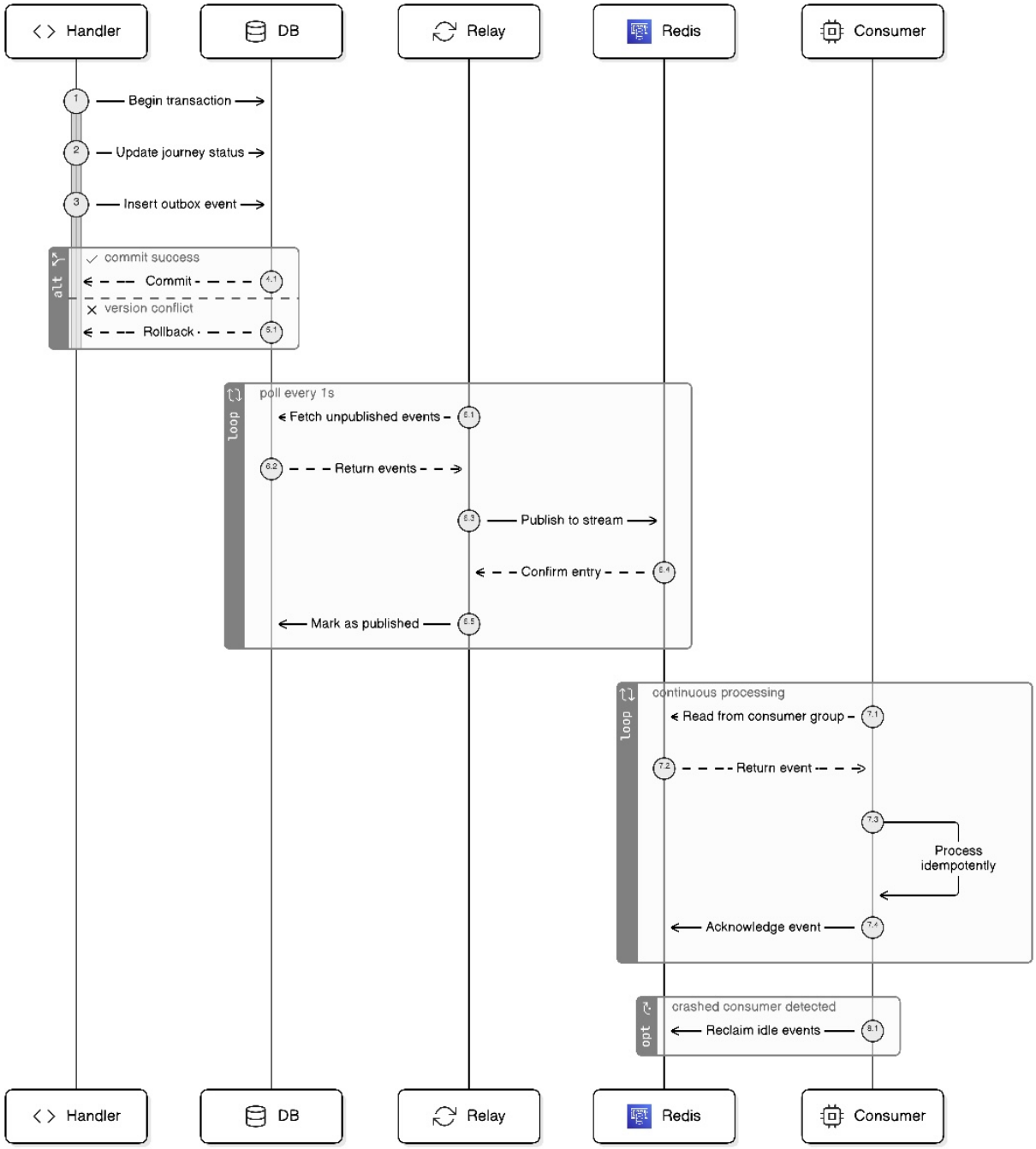


Figure 12: Transactional outbox pattern

The naive approach of calling Redis directly after `COMMIT` creates a crash window where state transitions commit but events are never published - leading to permanent slot leakage. The outbox writes an outbox row in the same transaction as the business record. A relay goroutine polls every 1 s for `published=false` rows, calls `XADD`, and sets `published=true` only on success. On restart it replays all unpublished rows. Consumer `UNIQUE(event_id)` constraints make redelivery idempotent.

4.2 Journey State Machine

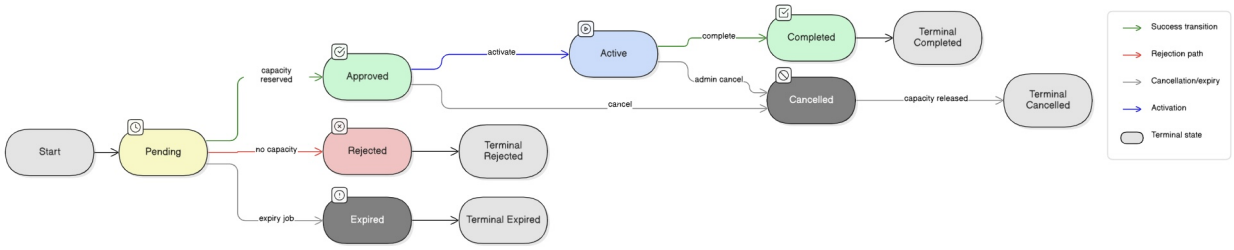


Figure 13: Journey state machine: seven states; three terminal. Slot release triggered on every terminal transition via Redis Stream event.

Enforcement at three independent layers: application code, database enum, and optimistic version locking (`WHERE id=$1 AND version=$expected`).

4.3 Idempotency

Every mutating endpoint accepts an `Idempotency-Key`. The server stores the key with the HTTP status and response body on first use; subsequent requests with the same key return the stored response. The Capacity service checks the cache twice - outside the transaction (fast path) and inside with `SELECT FOR UPDATE` (race-safe backstop) - ensuring at most one reservation row per key.

4.4 Orphan Cleanup and Follower Reads

If the Journey service crashes after Capacity commits a reservation but before the journey record is written, the capacity is permanently held. An orphan cleanup job in the Capacity service runs every 5 minutes and releases stale active reservations past their time window.

Dashboard queries use `AS OF SYSTEM TIME follower_read_timestamp()` to read from a local CockroachDB replica, reducing cross-region latency and leaseholder load. Bounded staleness of a few seconds is acceptable for dashboards but not for admission control.

```
SELECT segment_id, COALESCE(SUM(slots_used), 0.0)
FROM capacity.reservations AS OF SYSTEM TIME follower_read_timestamp()
WHERE status = 'active'
  AND time_window_start <= $1 AND time_window_end > $1
GROUP BY segment_id;
```

4.5 Service Startup and Graceful Degradation

Services apply schema migrations at startup using an idempotent migration ledger table, so restarts and redeploys do not require a separate manual migration step. Redis is treated as an optional accelerator at boot: if unavailable, services continue in reduced mode (no cache / delayed event fan-out) while preserving core correctness through the database path.

5. Testing

5.1 Test Strategy

Four layers: (1) **Unit tests** - business logic without external dependencies; (2) **Integration tests** - `reservation_distributed_test.go` spawns concurrent goroutines competing for the last slot on a shared segment, directly validating SERIALIZABLE isolation and lock ordering; (3) **End-to-end tests** - `TestProductionGradeE2ESystemF` runs against live GCP cells over HTTPS; (4) **Load and chaos tests** - K6 and `chaos-monkey.sh`.

5.2 E2E Scenario Coverage

Scenario	Assertion
Happy path booking	PENDING → APPROVED; capacity reduces; push notification delivered
Duplicate request	Same idempotency key returns cached response; exactly one reservation row
At-capacity rejection	REJECTED with `at_capacity` reason code
Segment closure	REJECTED with `segment_closed` reason code
Cancellation/release	Reservation transitions to released after cancel
Expiry job	APPROVED with past departure → EXPIRED within 90 s
Admin force-cancel	Admin JWT cancels any driver's journey; audit event recorded
Token refresh	Expired token triggers `/auth/refresh`; new token accepted by all services
Multi-cell isolation	EU booking not visible to US admin list
Concurrent bookings	10 simultaneous requests for same segment; exactly `max_capacity` succeed

Table 5: E2E test scenario coverage

5.3 Load Testing Results

K6 tests ran against all three live GCP cells on 2026-04-10 (`20260410T102719Z` artifact).

Cell	Test	Reqs	Errors	p95 (s)	p99 (s)	Notable
EU	Smoke	32	0.00%	1.10	1.39	Zero errors; journey p50=1.12 s
EU	Load	6988	0.34%	4.23	7.01	24 journey 502s (circuit-breaker)
EU	Stress	8675	11.25%	30.0	30.2	IAM pool saturated; 34% register err
US	Smoke	18	0.00%	6.41	7.48	WAN penalty: journey p50=6.9 s
US	Load	4993	11.62%	5.70	11.6	256 journey 502s; map 64% error
US	Stress	7731	36.58%	30.2	30.3	Map 100% err; Rate Limiting
APAC	Smoke	16	6.25%	7.25	11.2	Journey 502; route p50=5.9 s
APAC	Load	4127	10.56%	6.89	12.4	Journey 100% 502; map 44% error
APAC	Stress	5217	50.74%	-	-	Register 77% err; map 100% error

Table 6: K6 live-cell summary results (smoke/load/stress)

5.4 Chaos Testing

The `chaos-monkey.sh` script implements an eight-experiment suite across three phases. **Phase 1** (service-level) scales each of the four application services (`capacity-service`, `map-service`, `iam-service`, `notification-service`) to zero replicas by injecting an impossible Swarm placement constraint, verifying that nginx returns HTTP 502 within one poll interval while all other cells remain healthy. **Phase 2** (infrastructure) targets Redis and CockroachDB: scaling Redis to zero confirmed graceful cache-miss fallback to the database with no client-visible errors; scaling one CockroachDB node confirmed that the three-node quorum absorbed writes without downtime (a pre-flight quorum guard via `cockroach node status` blocks the experiment if fewer than three nodes are live). **Phase 3** (node-level) runs a full Swarm drain of the worker VM (`vcs-vm-eu2`) and a GCP deny-ingress firewall block against the same VM; in both cases the manager node (`vcs-vm-eu1`) continued serving all traffic with zero errors reported by the remaining cells. Safety mechanisms include: pre-flight health checks requiring all three regional cells to be healthy before any experiment begins; a Bash `EXIT` trap that executes a LIFO cleanup stack (constraint removal, firewall rule deletion, node re-activation) on any exit path including signals; `-detach` on all `docker service update` calls to prevent indefinite blocking; a per-experiment abort threshold that halts the suite if a non-target cell degrades; and dry-run mode for validation without live impact. All eight experiments passed, with drain-node and block-firewall both recording 100% availability in the surviving cell.

5.5 Observability

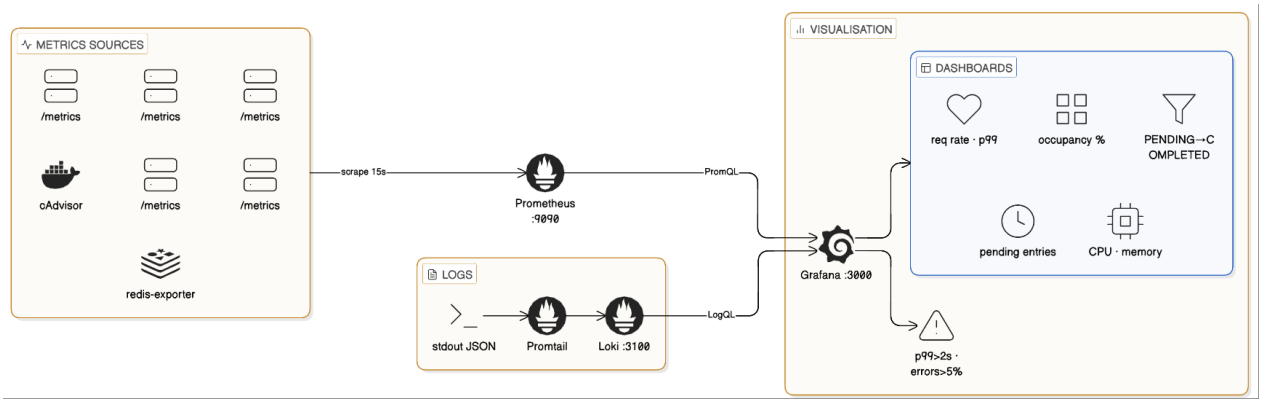


Figure 14: Observability stack

6. Allocation of Work

Member	Contributions
Deepika Nag	RS256 JWT issuance and JWKS publication; bcrypt cost-12 hashing; dual-token lifecycle with rotation; role-based JWT middleware; user profile management; RSA key rotation via `previous_kid`; admin promote and force-logout endpoints; `auth` schema migrations.
Ajinkya Taranekar	Three-phase booking orchestration; journey state machine with optimistic version locking; transactional outbox and relay goroutine; expiry job; idempotency cache; Map client with coordinate-to-node resolution; journey events audit table; admin journey endpoints; Docker Swarm stack; GCP provisioning; GitHub Actions CI/CD pipeline; E2E test suite; K6 load testing.
Jai Nagle	SERIALIZABLE reservation transaction with `SELECT FOR UPDATE` and alphabetic lock ordering; vehicle-type slot weighting; segment closure management; 30-second Redis availability cache; orphan cleanup job; Redis Streams consumer for slot release; `XAUTOCLAIM` idle-message reclaim; capacity occupancy endpoint; `capacity` schema.
Xiaoxuan Duan	In-memory directed graph; Dijkstra shortest-path; route cache (in-memory + Redis 1-hour TTL); segment ID alignment with Capacity; live traffic overlay endpoint; Nginx gateway configuration; frontend booking form with node selection and route preview; `map` schema.
Ziwei Zhao	Redis Streams consumer group with `XAUTOCLAIM`; Firebase Admin SDK FCM dispatch; delivery status state machine; stale token deactivation; device token registration; in-app notification list with read state; notification badge in frontend; `notification` schema.

Table 7: Work allocation by team member

Shared: Docker Compose local dev environment; observability stack configuration (Prometheus, Loki, Promtail, Grafana alert rules); chaos testing framework; cross-service integration fixes (segment ID alignment, Redis stream key consistency, Nginx routing corrections).

7. Summary

7.1 What Was Achieved

Clearway is a working globally-distributed system enforcing hard resource limits under concurrent write pressure. Verified properties:

- **No over-booking.** SERIALIZABLE isolation with alphabetically-ordered `SELECT FOR UPDATE` verified under ten simultaneous concurrent booking requests; no run exceeded capacity.
- **No event loss.** Transactional outbox verified by killing the relay mid-flight; all unpublished rows replayed correctly on restart without duplication.

- **Single-node failure tolerance.** Draining one Swarm node produced zero application errors in remaining cells.
- **Idempotency.** Duplicate booking requests produce exactly one reservation row.

7.2 Design Decisions in Retrospect

The cell-based architecture was the right choice: removing cross-region service calls simplified failure analysis and the no-global-capacity-view trade-off was acceptable. CockroachDB was vindicated by chaos testing. The transactional outbox was the most important correctness mechanism; without it, fire-and-forget publishing would lead to silent slot leakage under any process crash. The Saga coordinator for cross-region reservations was implemented and proved the right approach: it avoids the WAN-round-trip lock-escalation cost of a single distributed transaction spanning multiple leaseholders, while providing crash-safe compensation on partial failure. The main remaining limitation is the in-process nature of background jobs (outbox relay, expiry job); in production these would become separate deployable units.

7.3 Production Upgrade Path

The architecture intentionally keeps replacement boundaries clean:

- Kubernetes + HPA can replace Swarm without changing service contracts.
- Kafka/Redpanda can replace Redis Streams if stronger cross-DC log semantics are needed.
- gRPC can replace HTTP/JSON for lower overhead intra-cell calls.
- Managed distributed SQL can replace self-managed CockroachDB with minimal code change.

7.4 Conclusion

The four mechanisms doing the heaviest correctness lifting are: (1) SERIALIZABLE transaction with ordered lock acquisition, which prevents over-booking under concurrent writes; (2) the Saga coordinator, which makes cross-region reservations crash-safe through per-region compensation rather than a single distributed lock spanning WAN leaseholders; (3) the transactional outbox, which prevents silent event loss under process crash; and (4) JWKS-based decentralised token verification, which decouples authentication availability from every other service. The Map service's dual-mode architecture - static in-memory Dijkstra for named nodes, live OSRM for arbitrary GPS coordinates with DB-persistent caching - allows the system to serve both pre-seeded and real-world routes without coupling correctness to external API availability. None of these required novel algorithms; they are standard distributed systems patterns applied deliberately to this domain. The load tests made the CAP trade-off visible in measured milliseconds: strong consistency across regions is not optional, and its latency cost is the expected price.